



Lowerbounds & Polynomial Multiplication

Lecture 3 | Algorithms Design & Analysis

Girish Varma
IIIT Hyderabad

How fast can Multiplication get?



We saw multiplication of two n digit numbers can be done in $o(n^2)$ (Small oh), to be precise $\Theta(n^{1.6...})$ using Karatsuba's Algorithm.

This raises the question of whether it can be done in $o(n^{1.6...})$ or even $o(n)$

$\Omega(n)$ Lower Bound



It cannot be done in $o(n)$. Why?

The algorithm has to at least write the output which is n digits long.

So the complexity is $\Omega(n)$.

Sorting



Using merge sort sorting of an array of n numbers (numbers here are assumed to be fixed number of bits, say 8 or 32 bits).

Complexity = $\Theta(n \log n)$

$\Omega(n \log n)$ lower bound



Read page 59 in

<http://algorithmics.isi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>

Polynomial Multiplication Problem



We have so far seen how divide-and-conquer gives fast algorithms for multiplying integers and matrices; our next target is *polynomials*. The product of two degree- d polynomials is a polynomial of degree $2d$, for example:

$$(1 + 2x + 3x^2) \cdot (2 + x + 4x^2) = 2 + 5x + 12x^2 + 11x^3 + 12x^4.$$

More generally, if $A(x) = a_0 + a_1x + \cdots + a_dx^d$ and $B(x) = b_0 + b_1x + \cdots + b_dx^d$, their product $C(x) = A(x) \cdot B(x) = c_0 + c_1x + \cdots + c_{2d}x^{2d}$ has coefficients

$$c_k = a_0b_k + a_1b_{k-1} + \cdots + a_kb_0 = \sum_{i=0}^k a_ib_{k-i}$$

Algorithm



For each coefficient c_i , equation has around $O(d)$ terms. There are $2d$ coefficients for c .

Compute all equations costs: $\Theta(d^2)$

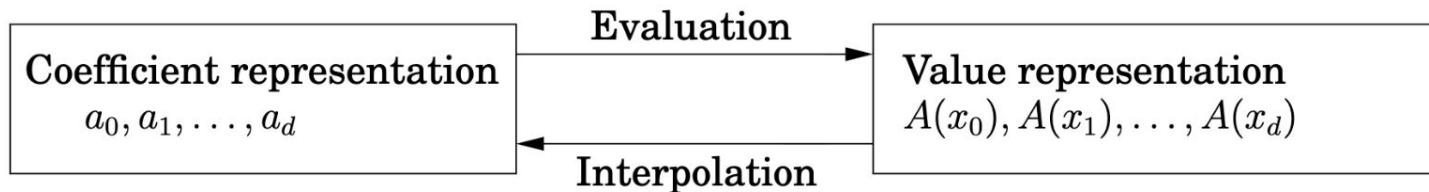
Alternative Polynomial Representation



Fact A degree- d polynomial is uniquely characterized by its values at any $d + 1$ distinct points.

A familiar instance of this is that “any two points determine a line.” We will later see why the more general statement is true (page 72), but for the time being it gives us an *alternative representation* of polynomials. Fix any distinct points $x_0, \dots, x_d$. We can specify a degree- d polynomial $A(x) = a_0 + a_1x + \dots + a_dx^d$ by either one of the following:

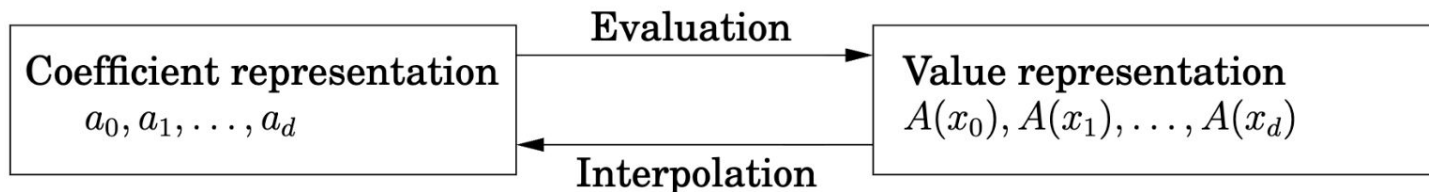
1. Its coefficients $a_0, a_1, \dots, a_d$
2. The values $A(x_0), A(x_1), \dots, A(x_d)$



Algorithm Idea



Since the product $C(x)$ has degree $2d$, it is completely determined by its value at any $2d + 1$ points. And its value at any given point z is easy enough to figure out, just $A(z)$ times $B(z)$. Thus *polynomial multiplication takes linear time in the value representation*.



Alternative Algorithm



Figure 2.5 Polynomial multiplication

Input: Coefficients of two polynomials, $A(x)$ and $B(x)$, of degree d

Output: Their product $C = A \cdot B$

Selection

Pick some points $x_0, x_1, \dots, x_{n-1}$, where $n \geq 2d + 1$

Evaluation

Compute $A(x_0), A(x_1), \dots, A(x_{n-1})$ and $B(x_0), B(x_1), \dots, B(x_{n-1})$

Multiplication

Compute $C(x_k) = A(x_k)B(x_k)$ for all $k = 0, \dots, n - 1$

Interpolation

Recover $C(x) = c_0 + c_1x + \dots + c_{2d}x^{2d}$

Problem: Evaluation itself takes $O(d^2)$

Idea for faster evaluation



Here's an idea for how to pick the n points at which to evaluate a polynomial $A(x)$ of degree $\leq n - 1$. If we choose them to be positive-negative pairs, that is,

$$\pm x_0, \pm x_1, \dots, \pm x_{n/2-1},$$

then the computations required for each $A(x_i)$ and $A(-x_i)$ overlap a lot, because the even powers of x_i coincide with those of $-x_i$.

Idea for faster evaluation



$$A(x) = A_e(x^2) + xA_o(x^2),$$

where $A_e(\cdot)$, with the even-numbered coefficients, and $A_o(\cdot)$, with the odd-numbered coefficients, are polynomials of degree $\leq n/2 - 1$ (assume for convenience that n is even). Given *paired* points $\pm x_i$, the calculations needed for $A(x_i)$ can be recycled toward computing $A(-x_i)$:

$$\begin{aligned} A(x_i) &= A_e(x_i^2) + x_i A_o(x_i^2) \\ A(-x_i) &= A_e(x_i^2) - x_i A_o(x_i^2). \end{aligned}$$

$$3 + 4x + 6x^2 + 2x^3 + x^4 + 10x^5 = (3 + 6x^2 + x^4) + x(4 + 2x^2 + 10x^4).$$

$$\begin{aligned} A_e(x) &= 3 + 6x + x^2 \\ A_o(x) &= 4 + 2x + 10x^2 \end{aligned}$$

Idea for faster evaluation



In other words, evaluating $A(x)$ at n paired points $\pm x_0, \dots, \pm x_{n/2-1}$ reduces to evaluating $A_e(x)$ and $A_o(x)$ (which each have half the degree of $A(x)$) at just $n/2$ points, $x_0^2, \dots, x_{n/2-1}^2$.

Evaluate:

$A(x)$
degree $\leq n - 1$

at:

$+x_0 \quad -x_0 \quad +x_1 \quad -x_1 \quad \dots \quad +x_{n/2-1} \quad -x_{n/2-1}$

Equivalently,
evaluate:

$A_e(x)$ and $A_o(x)$
degree $\leq n/2 - 1$

at:

$x_0^2 \quad x_1^2 \quad \dots \quad x_{n/2-1}^2$

Idea for faster evaluation



The original problem of size n is in this way recast as two subproblems of size $n/2$, followed by some linear-time arithmetic. If we could recurse, we would get a divide-and-conquer procedure with running time

$$T(n) = 2T(n/2) + O(n),$$

which is $O(n \log n)$, exactly what we want.

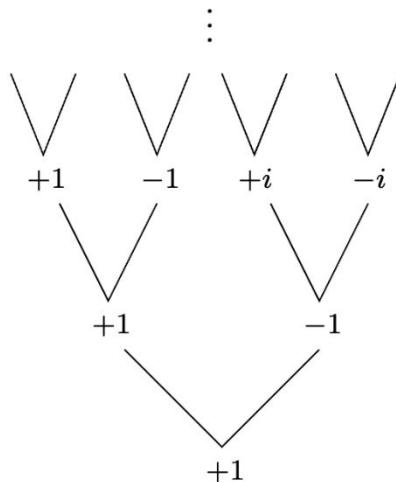
But we have a problem: The plus-minus trick only works at the top level of the recursion. To recurse at the next level, we need the $n/2$ evaluation points $x_0^2, x_1^2, \dots, x_{n/2-1}^2$ to be *themselves* plus-minus pairs. But how can a square be negative? The task seems impossible!

Great Idea II



we use complex numbers.

Great Idea II: But which?

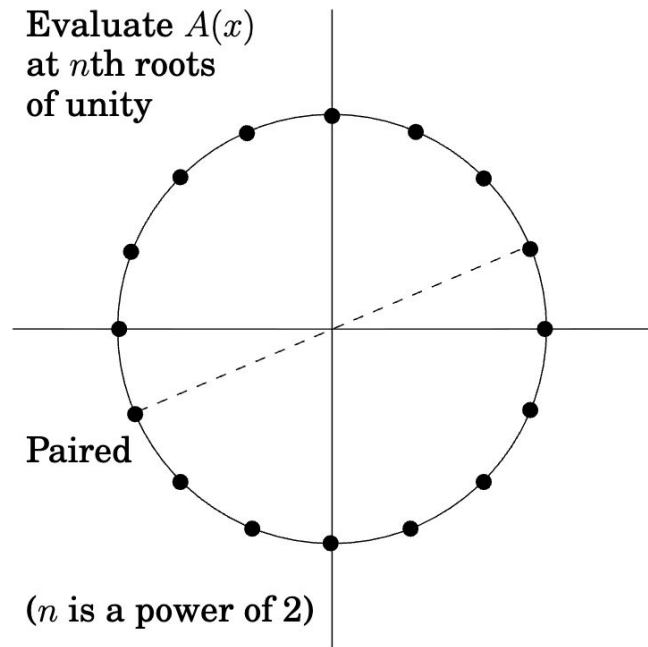


The next level up then has $\pm\sqrt{+1} = \pm 1$ as well as the *complex* numbers $\pm\sqrt{-1} = \pm i$, where i is the imaginary unit. By continuing in this manner, we eventually reach the initial set of n points. Perhaps you have already guessed what they are: the *complex n th roots of unity*, that is, the n complex solutions to the equation $z^n = 1$.

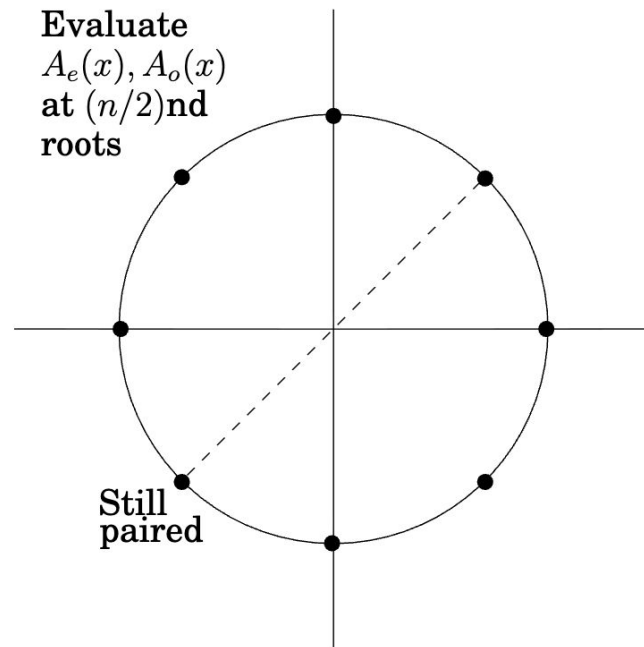
Divide and Conquer Algorithm



Divide-and-conquer step



Divide and
conquer

A black arrow pointing from the left diagram to the right diagram, indicating the "Divide and conquer" step.

Fast Fourier Transform



Figure 2.7 The fast Fourier transform (polynomial formulation)

function FFT(A, ω)

Input: Coefficient representation of a polynomial $A(x)$
of degree $\leq n-1$, where n is a power of 2
 ω , an n th root of unity

Output: Value representation $A(\omega^0), \dots, A(\omega^{n-1})$

if $\omega = 1$: return $A(1)$

express $A(x)$ in the form $A_e(x^2) + xA_o(x^2)$

call $\text{FFT}(A_e, \omega^2)$ to evaluate A_e at even powers of ω

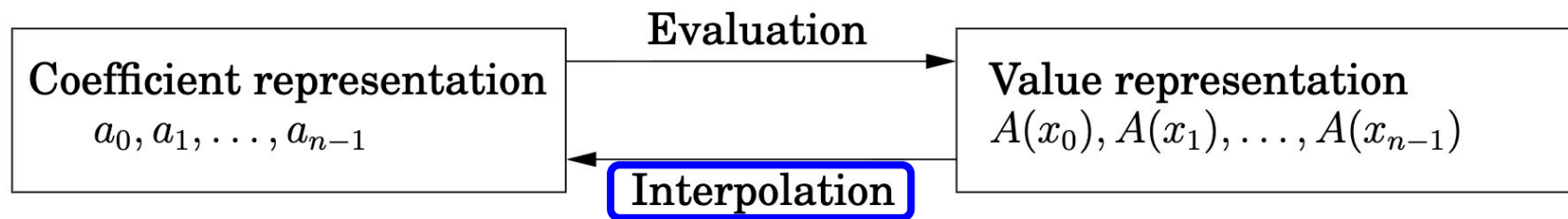
call $\text{FFT}(A_o, \omega^2)$ to evaluate A_o at even powers of ω

for $j = 0$ to $n-1$:

 compute $A(\omega^j) = A_e(\omega^{2j}) + \omega^j A_o(\omega^{2j})$

return $A(\omega^0), \dots, A(\omega^{n-1})$

What's Left?



Read



Chapter 2, Section 2.6 in

<http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>

Solve



Exercises 2.7, 2.8, 2.10, 2.29, in

<http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>